IL1332 Digital Design using HDL

L13: Moore and Mealy Finite State Machines (FSMs)

Ahmed Hemani

EECS/EE/EES, KTH, Stockholm, Sweden

Connecting to Text Book

3.4 Finite State Machines 123
3.4.1 *FSM Design Example* 123
3.4.2 *State Encodings* 129
3.4.3 *Moore and Mealy Machines* 132
3.4.4 *Factoring State Machines* 134
3.4.5 *Deriving an FSM from a Schematic* 137
3.4.6 *FSM Review* 140

© Ahmed Hemani, KTH, Stockholm

Moore FSM (Finite State Machines)

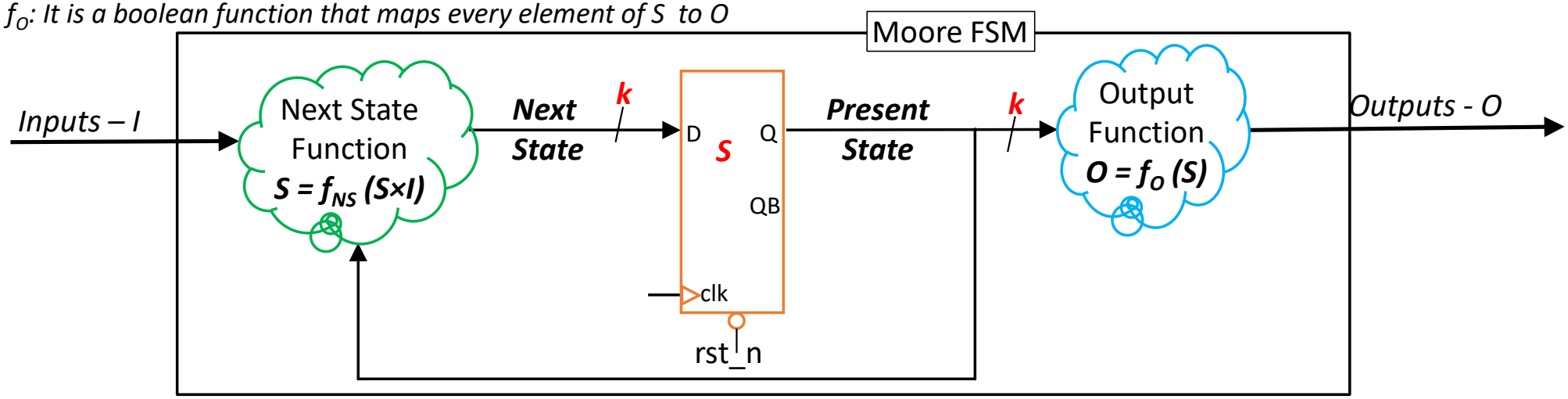
Edward F Moore (American mathematician)



1. *S*: Set of *P* States = $\{s_0, s_1, s_2 \ldots s_{P-1}\}$; *k: Number of Flops = round_up* $[\log_2(P)]$. *There are* $(2^k - P)$ *Illegal States*
2. $s_i \in S$: *Initial State. FSM after reset is in state* s_i.

3. *I: Set of Inputs* $\{i_0, i_1, i_2 \ldots i_{M-1}\}$. *If input I has 2 bits, I = {00, 01, 10, 11}*
4. *O: Set of Outputs* $\{o_0, o_1, o_2 \ldots i_{N-1}\}$. *If output O has 2 bits, O = {00, 01, 10, 11}*

5. f_{NS}: *It is a boolean function that maps every combination of I and S (S×I) to an element in set S*
   The element in set S that f_{NS} *computes is called* ***Next State****.*
   On the rising edge of the clock, it becomes ***Present State;*** *the S in S×I is the present state*
6. f_O: *It is a boolean function that maps every element of S to O*



1. Note that in Moore machines, outputs can ONLY change in response to changes in the states
2. Since states are synchronized with the clock, the outputs change synchronously

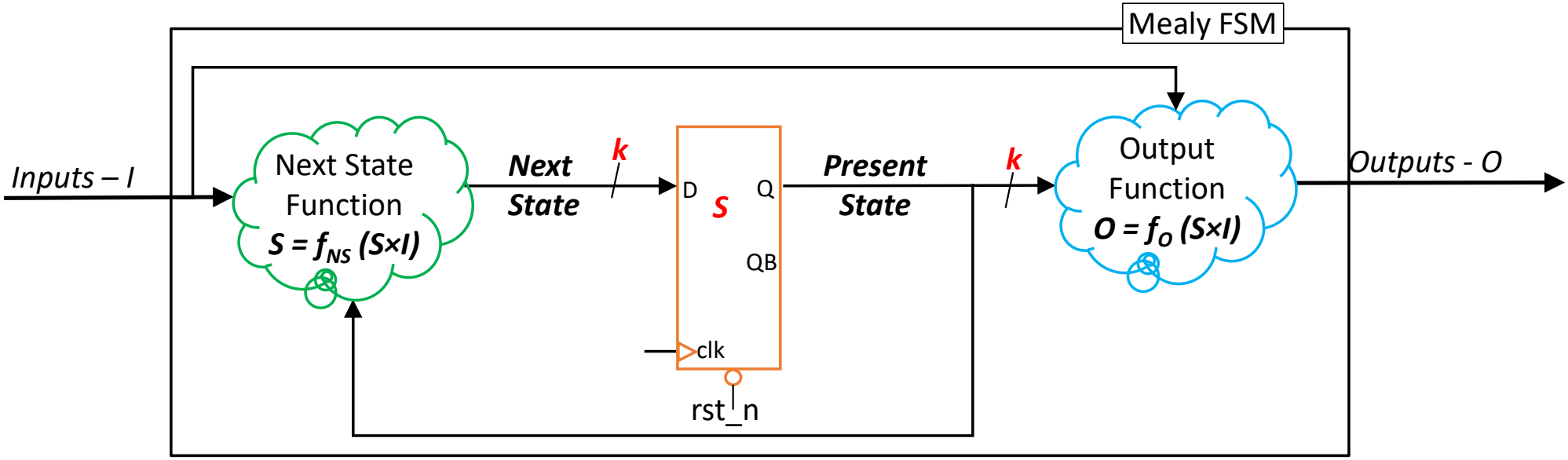
© Ahmed Hemani, KTH, Stockholm

Mealy FSM (Finite State Machines)

(George H Mealy - American mathematician)



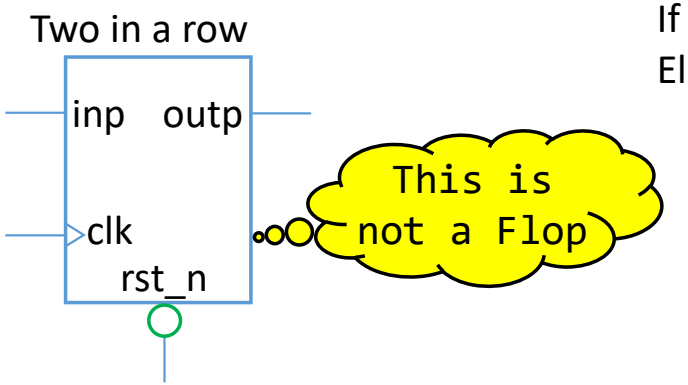
1. *f_O: It is a boolean function that maps every combination of S and I (S×I) to O*



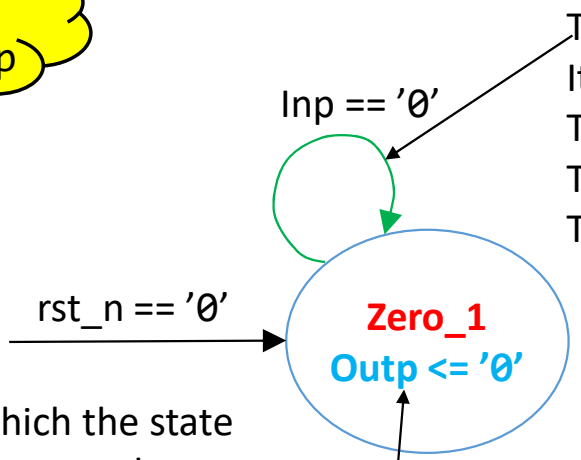
1. Note that in Mealy machines, outputs can ALSO change in response to changes in the inputs
2. Since inputs are NOT synchronized with the clock, the outputs change a*synchronously*

© Ahmed Hemani, KTH, Stockholm

Two in a row example – Moore FSM – State Diagram



If input, inp= 1, for two or more clocks in a row, set outp =1
Else set outp = 0



This is a transition arc
It is labelled with a condition (N.B: "==" means equal to)
The condition says, stay in state **zero_1** as long as *inp=0*
The base of arc is present state.
The tip of arrow is next state

Zero_1: Is the symbolic name of the state

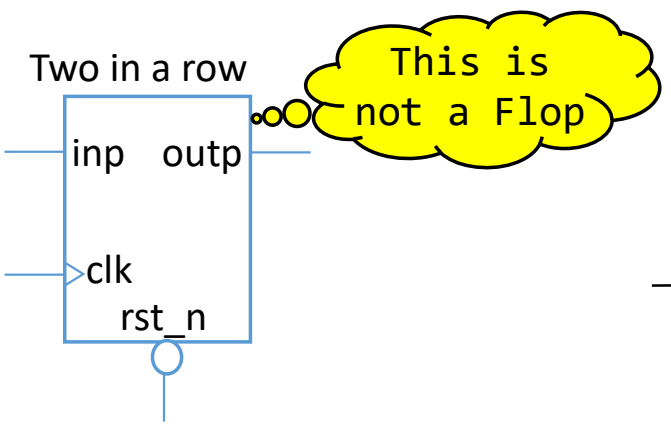
Zero_1: Serves the purpose of remembering that we have not seen any "inp==1" yet

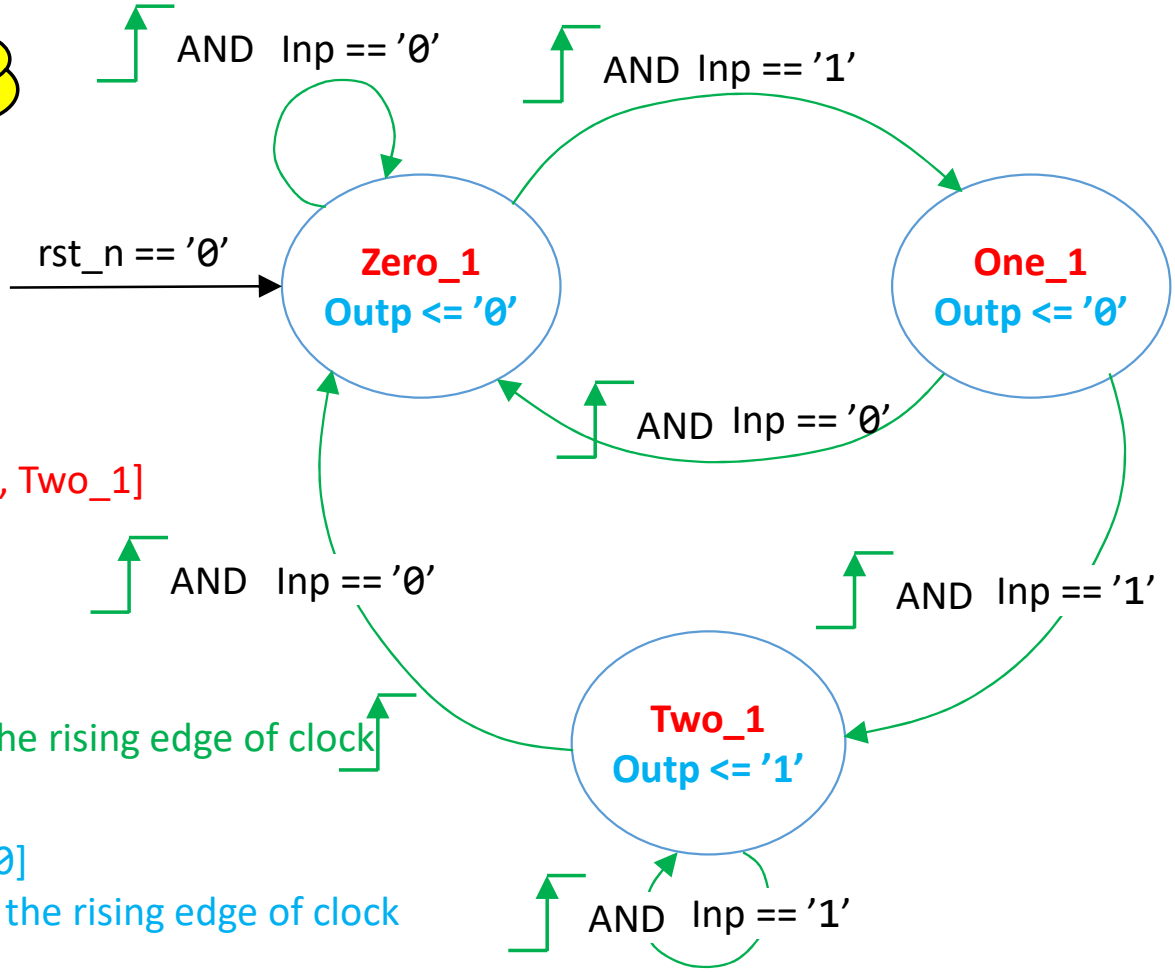
Zero_1 is the initial state in which the state machine starts when *rst_n* is asserted

In Moore FSMs, output is associated with a state
In **zero_1**, since we have not seen two 1s yet, we output 0, "**outp <= 0**"
N.B: "<=" means is assigned to

© Ahmed Hemani, KTH, Stockholm

Two in a row example – Moore FSM – State Diagram





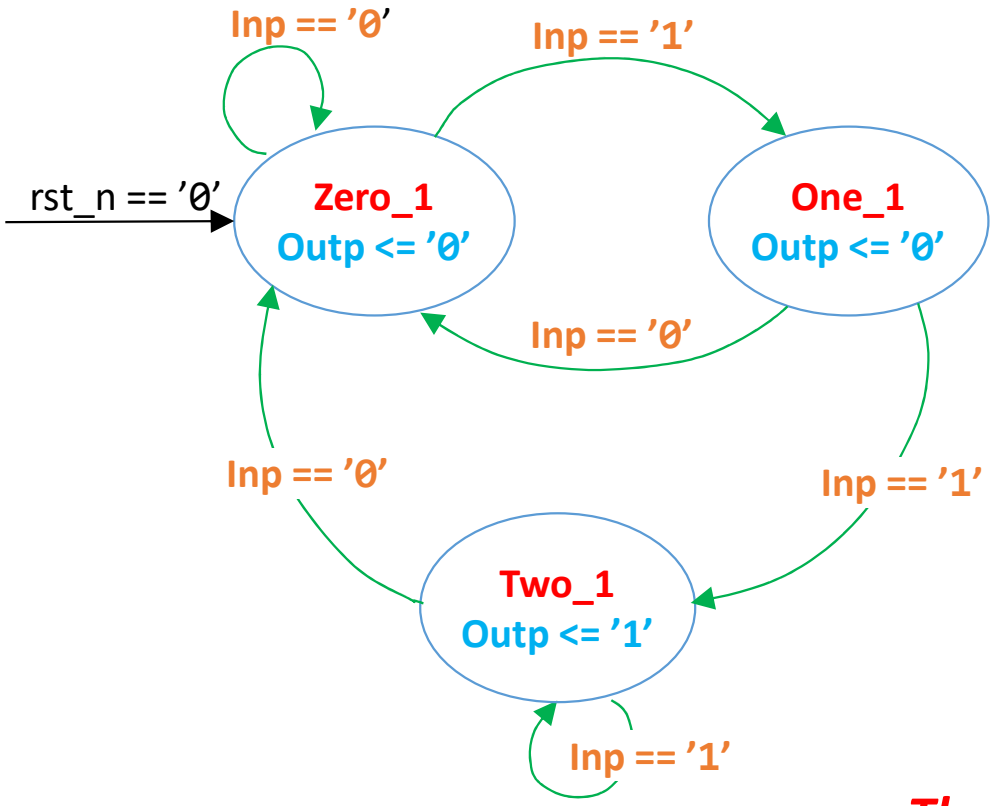
Symbolic State Names = [Zero_1, One_1, Two_1]

Transition Conditions = [inp == 1, inp == 0]
Every transition condition is evaluated on the rising edge of clock
And when rst_n is de-asserted

Output Assignment = [Outp = 1, Outp = 0]
Every output Assignment is executed on the rising edge of clock

© Ahmed Hemani, KTH, Stockholm

Two in a row example – Moore FSM – Transition Table

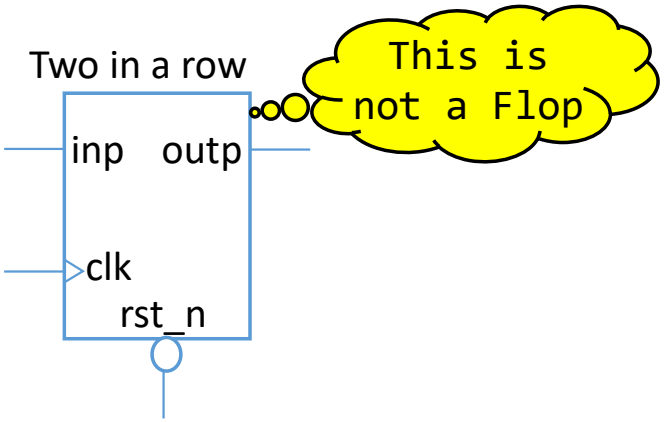


Input	Present State	Next State	Output
rst_n == 0	-	Zero_1	Outp <= 0
rst_n = 1	Inp == 0	Zero_1	Zero_1	Outp <= 0
rst_n = 1	Inp == 1	Zero_1	One_1	Outp <= 0
rst_n = 1	Inp == 0	One_1	Zero_1	Outp <= 0
rst_n = 1	Inp == 1	One_1	Two_1	Outp <= 0
rst_n = 1	Inp == 0	Two_1	Zero_1	Outp <= 1
rst_n = 1	Inp == 1	Two_1	Two_1	Outp <= 1

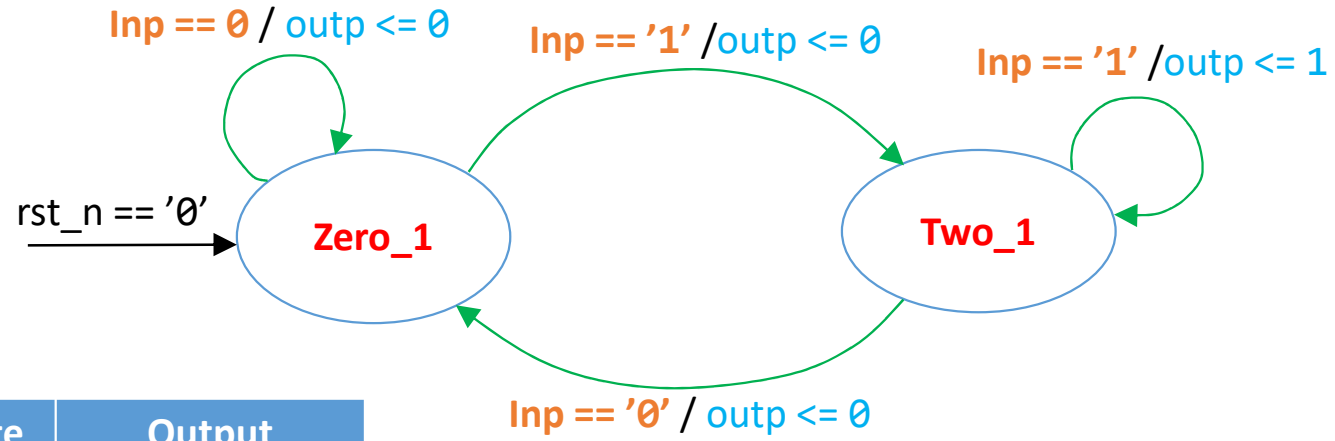
The Output is dependent only on the Present State

© Ahmed Hemani, KTH, Stockholm

Two in a row example – Mealy FSM



Mealy transition arcs are labelled as
Condition / output assignments



	Input	Present State	Next State	Output
	rst_n == 0	-	Zero_1	Outp <= 0
rst_n = 1	Inp == 0	Zero_1	Zero_1	Outp <= 0
	Inp == 1	Zero_1	Two_1	Outp <= 0
	Inp == 1	Two_1	Two_1	Outp <= 1
	Inp == 0	Two_1	Zero_1	Outp <= 0

Output is dependent on both Inputs and Present State

© Ahmed Hemani, KTH, Stockholm

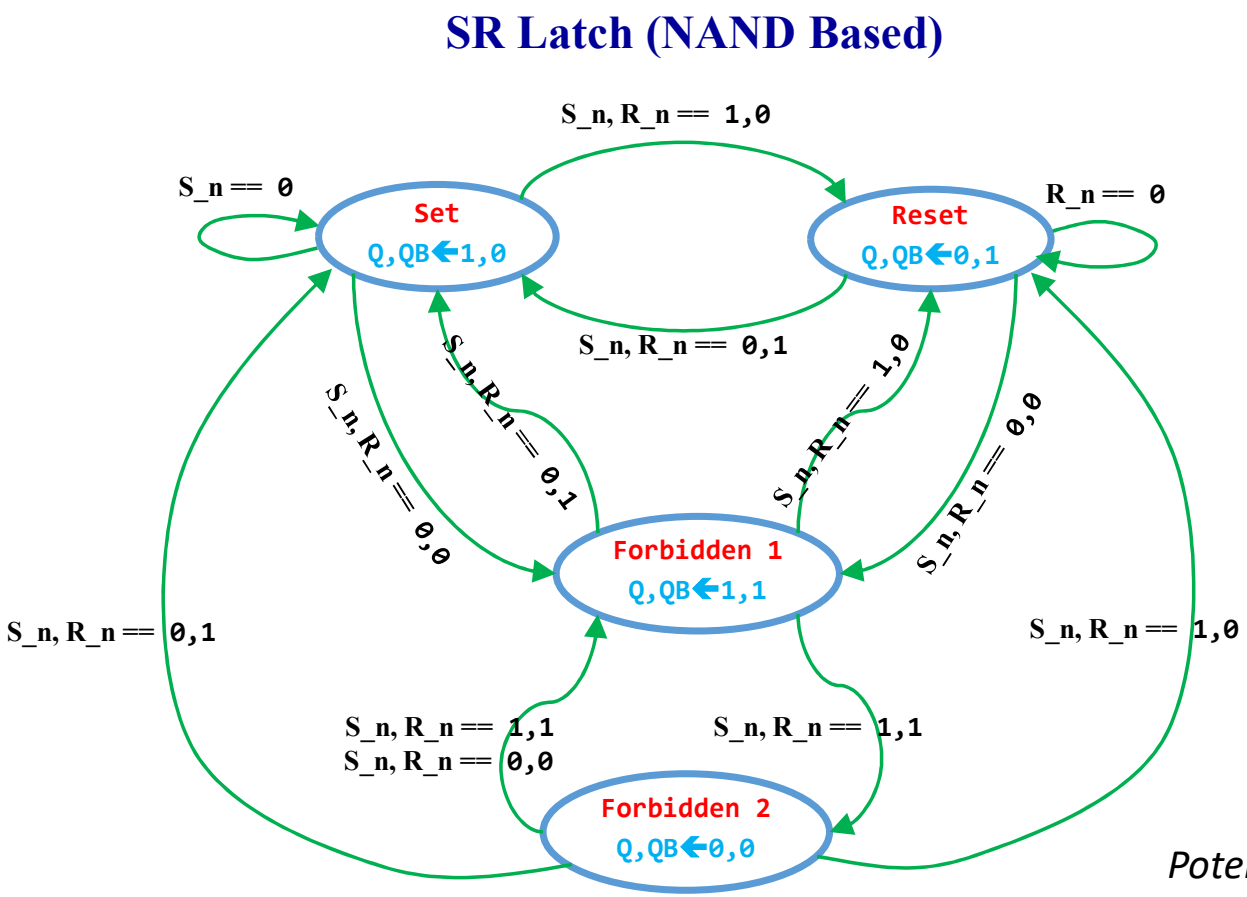
Quiz

Can you solve the “the two ones in a row” problem using combinational logic?

Answers:
A: Yes, it can be solved
B: No, it cannot be solved

© Ahmed Hemani, KTH, Stockholm

The Sequential Elements that we have studied are also FSMs



Quiz Time: Is this a
A: Moore Machine
B: Mealy Machine
C: Both

Potential Exam Question:
Draw State Diagram of D, J-K and T Flops

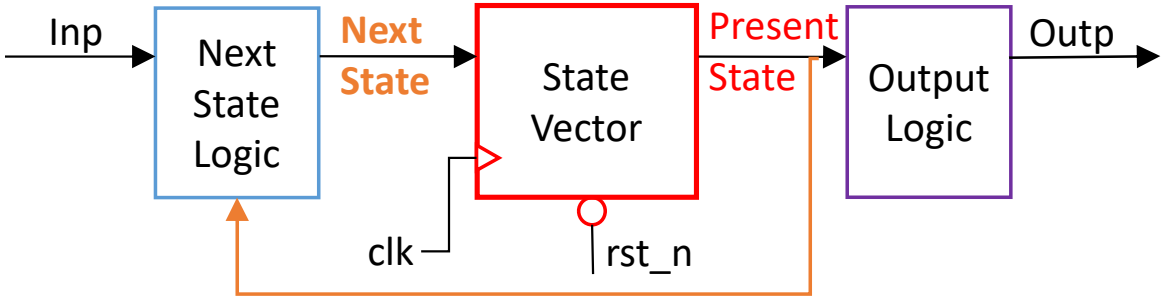
© Ahmed Hemani, KTH, Stockholm

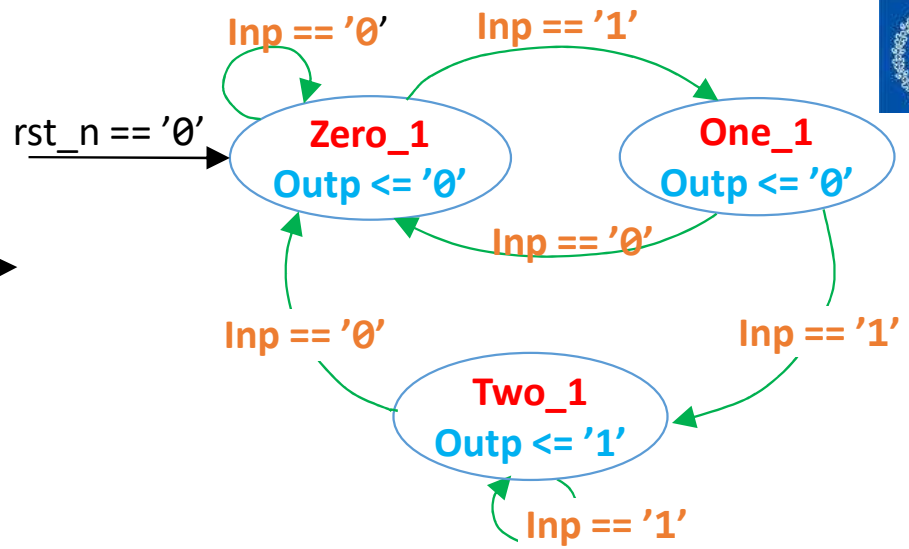
Moore vs Mealy

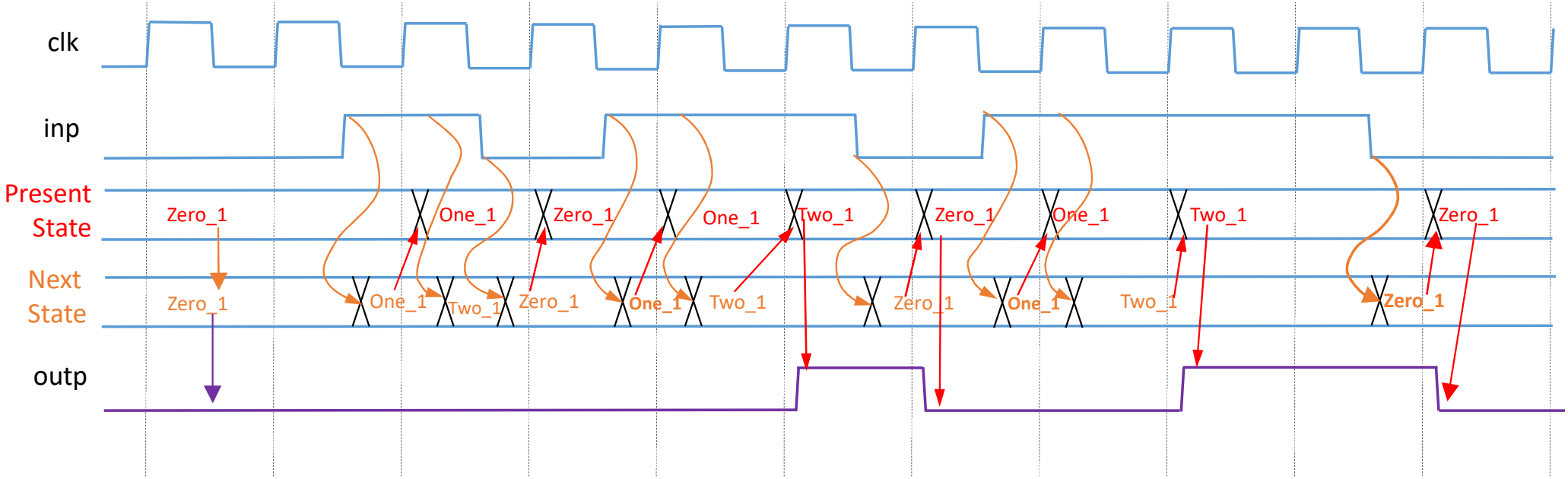
1. *Moore FSM's output depends only on the present state*
2. *Mealy FSM's output depends on the present state and the inputs*
3. *Mealy FSM often needs fewer states*
4. *Mealy FSM's outputs are not registered, for this reason an extra register is added*
5. *For most complex real life FSMs, it is common to adopt registered output Mealy Implementation Style. It is possible to have some states that are Moore and other that are Mealy*

© Ahmed Hemani, KTH, Stockholm

Moore FSM Timing



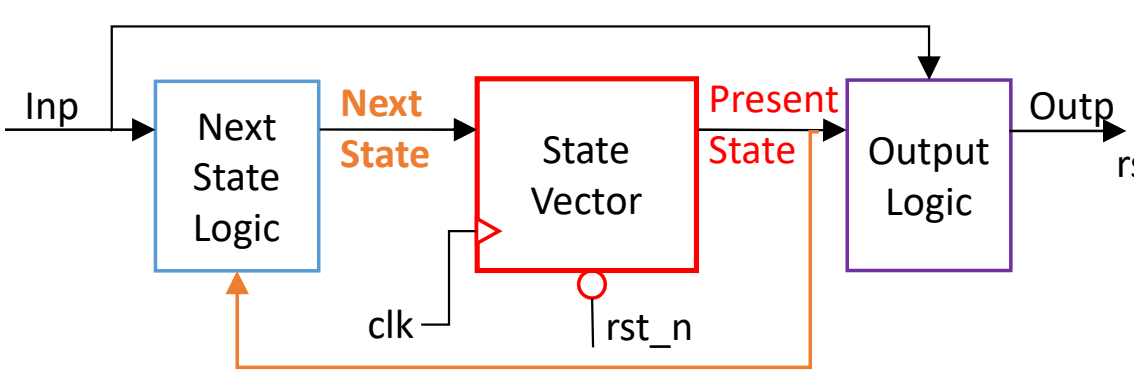


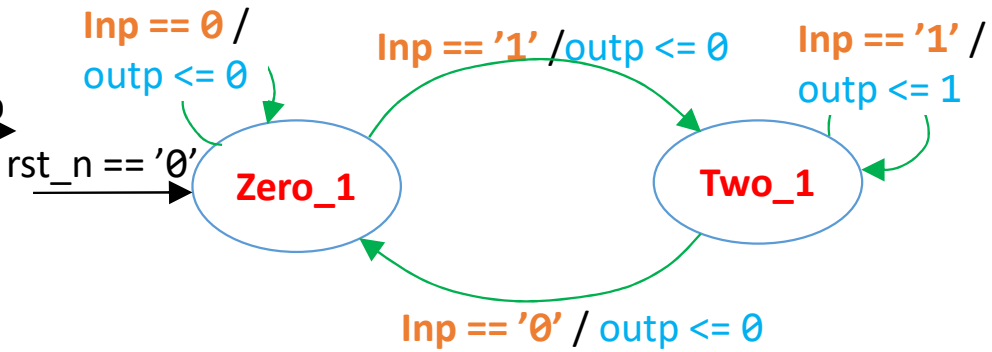


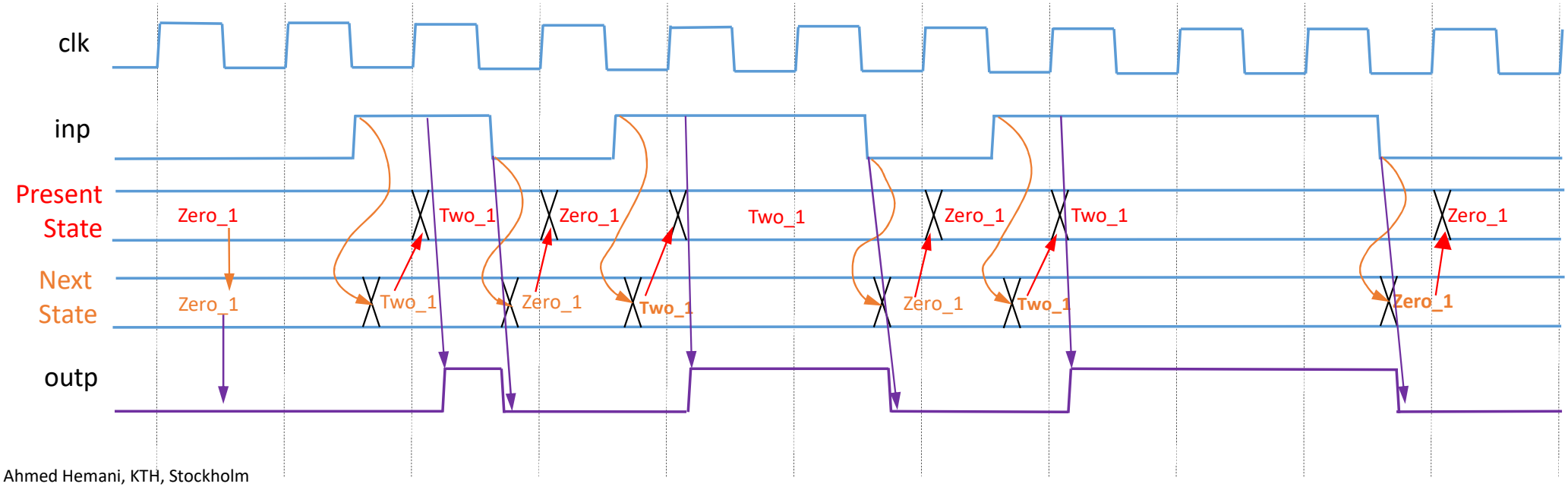
© Ahmed Hemani, KTH, Stockholm

Mealy FSM Timing

Mealy transition arcs are labelled as
Condition / output assignments

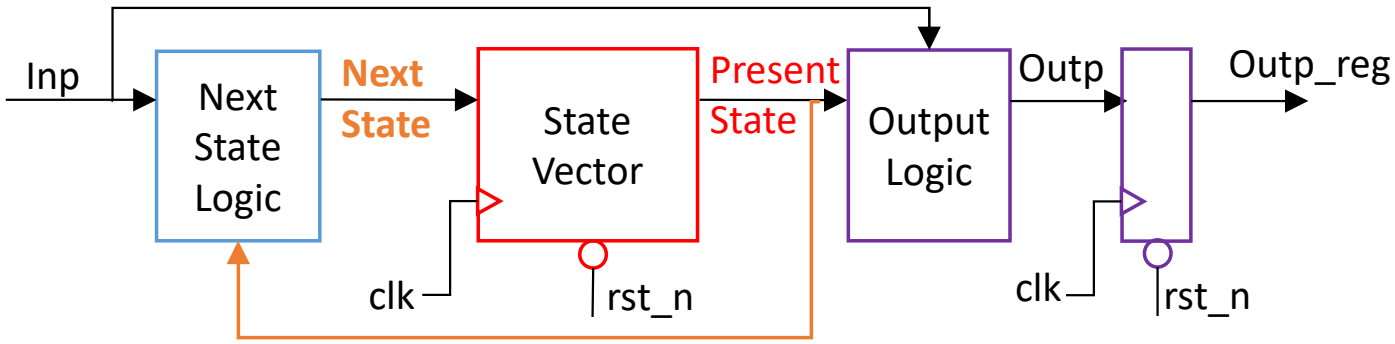


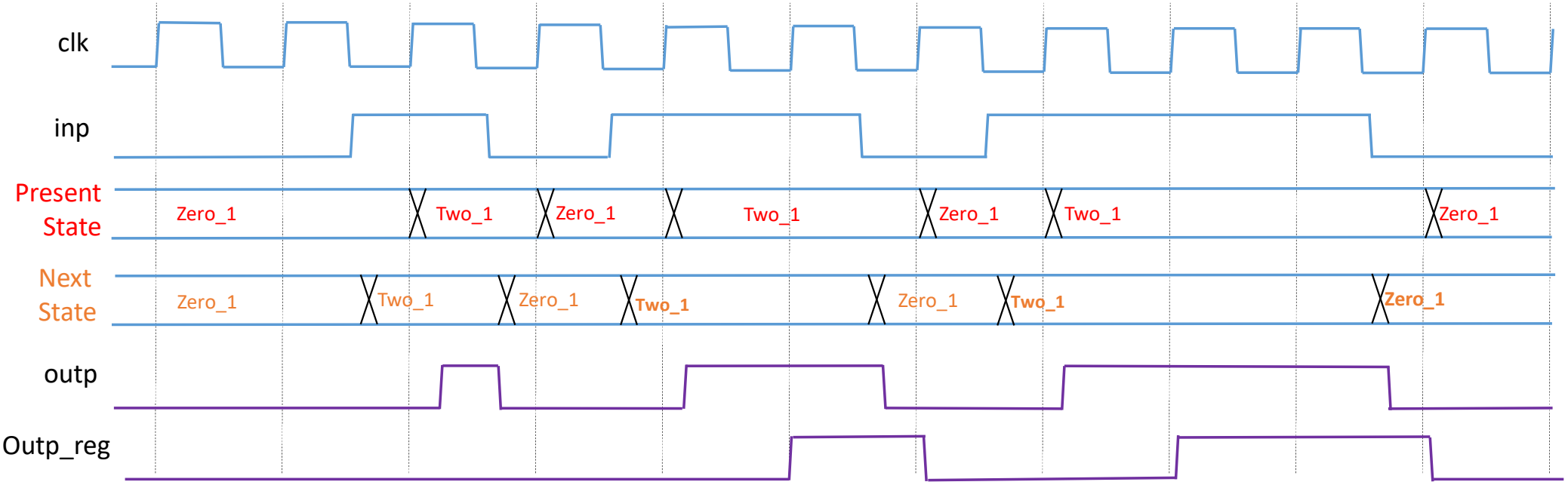




© Ahmed Hemani, KTH, Stockholm

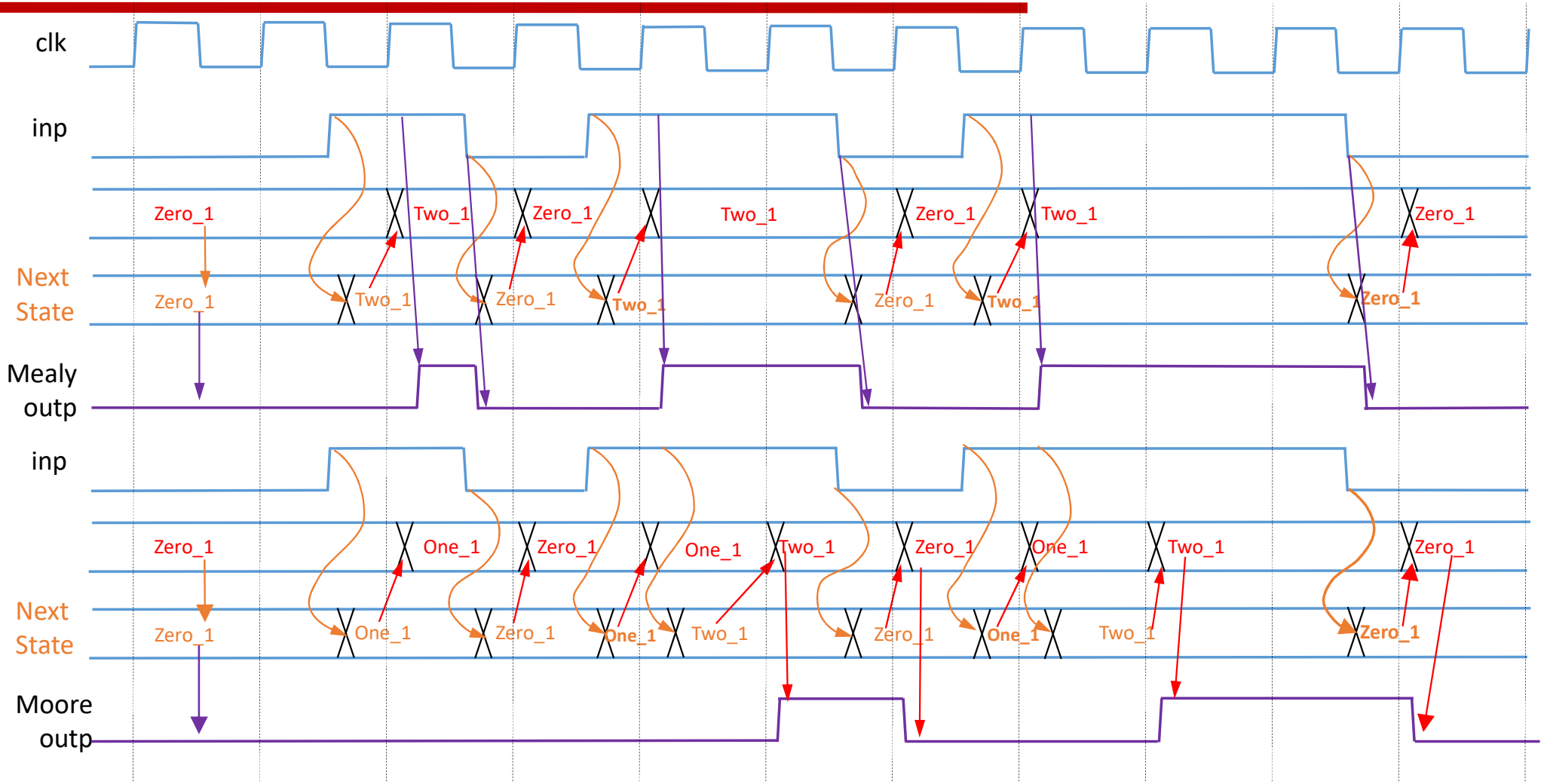
Registered Mealy FSM Outputs Solves the Asynchronous Outputs Problem





© Ahmed Hemani, KTH, Stockholm

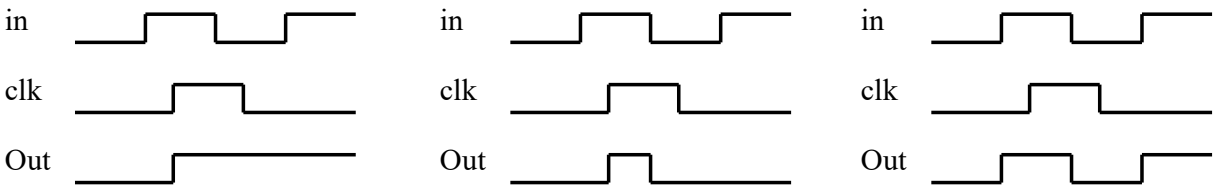
Moore vs. Mealy Timing



© Ahmed Hemani, KTH, Stockholm

Quiz on Moore FSMs

Which of the following exhibits behaviour of a Moore FSM ?



Alt 1 Alt 2 Alt 3



A: Alt 1
B: Alt 2
C: Alt 3
D: All the alternatives

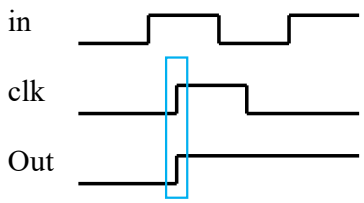
© Ahmed Hemani, KTH, Stockholm

Quiz on Moore FSMs

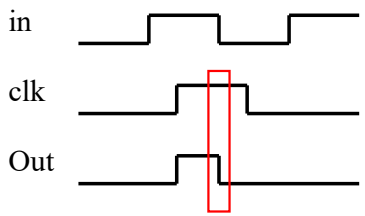
The reason why Alt 1 is a Moore FSM is because the ***out*** changes value synchronously with the clock (edge)

The reason why Alt 2 is not a Moore FSM is because the ***out*** changes value asynchronously with the clock edge

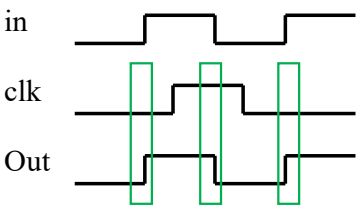
The reason why Alt 3 is not a Moore FSM is because the ***out*** changes value asynchronously with the clock edge



Alt 1



Alt 2

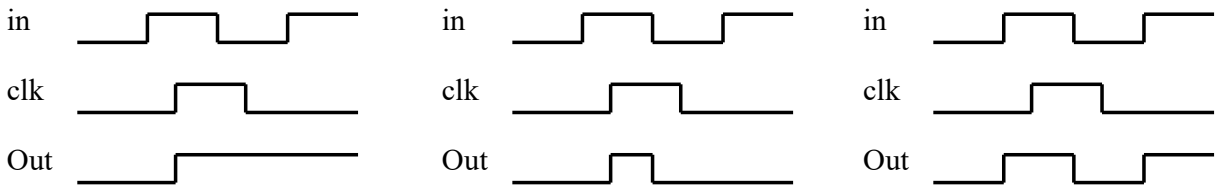


Alt 3

© Ahmed Hemani, KTH, Stockholm

Quiz on Mealy FSMs

Which of these are Mealy FSMs ?



Alt 1 Alt 2 Alt 3

A: Alt 1
B: Alt 2
C: Alt 3
D: All the alternatives

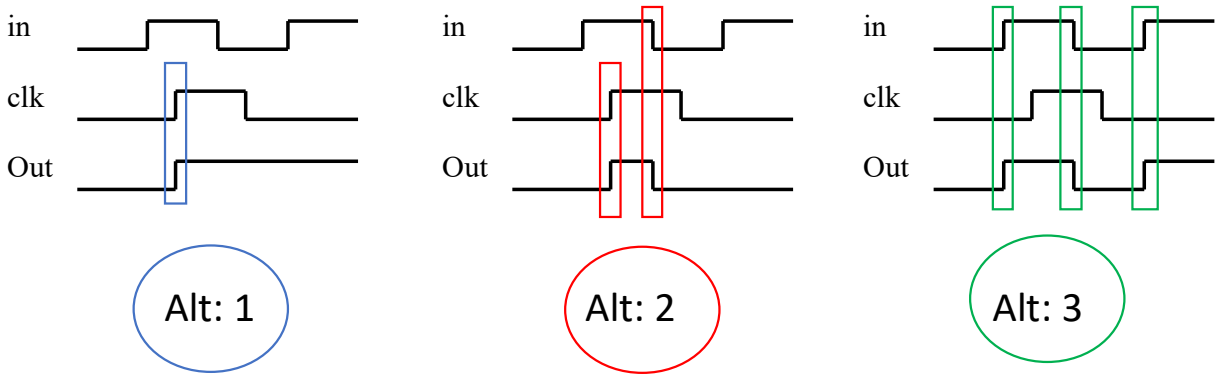
© Ahmed Hemani, KTH, Stockholm

Quiz on Mealy FSMs

*All of them are **potentially** Mealy FSM*

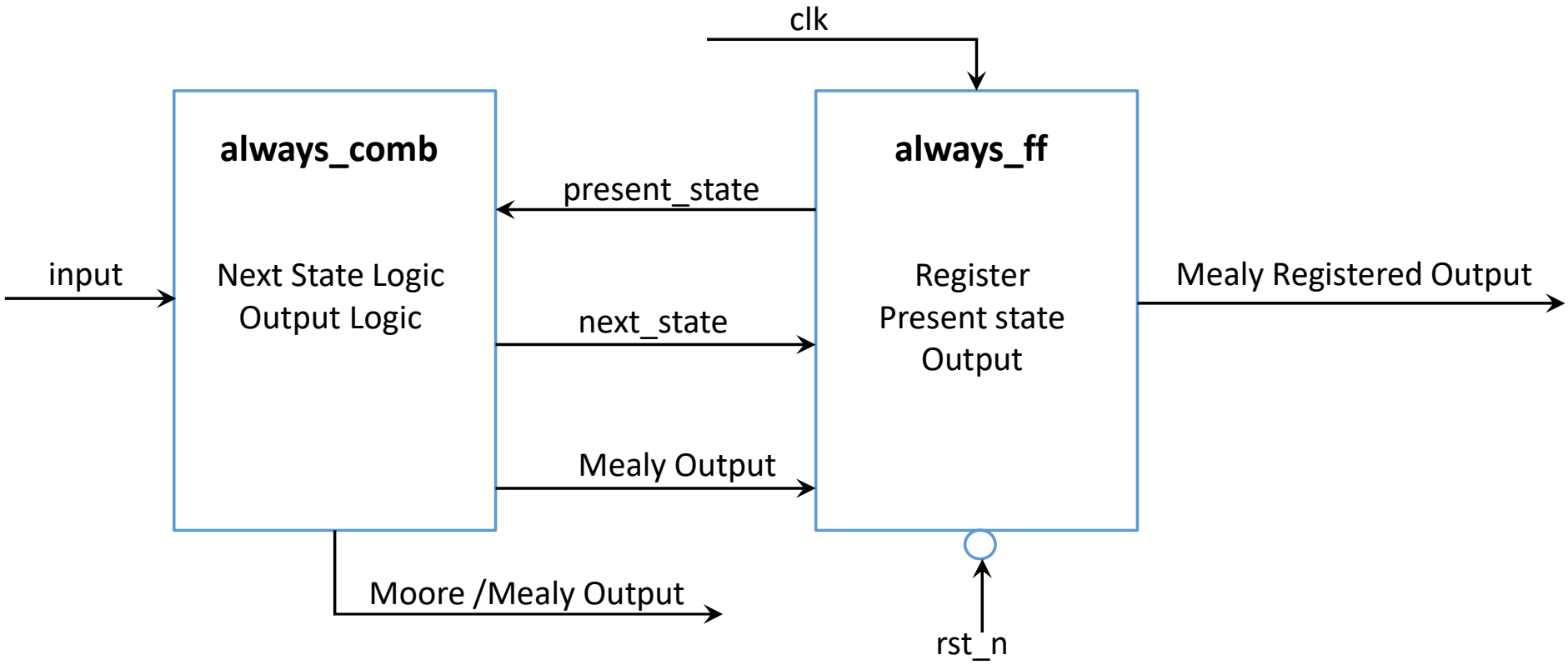
*Because the **outputs in Mealy FSM** are a function of both*

1. ***Present state** that changes synchronously with the clock edge and*
2. ***Input** that changes asynchronously with the clock edge*
   a. *In Alt 1, output is shown to change with clk, i.e., present state*
   b. *In Alt 2, output is shown to change with both clk (present state) and input*
   c. *In Alt 3, output is shown to change with inputs*

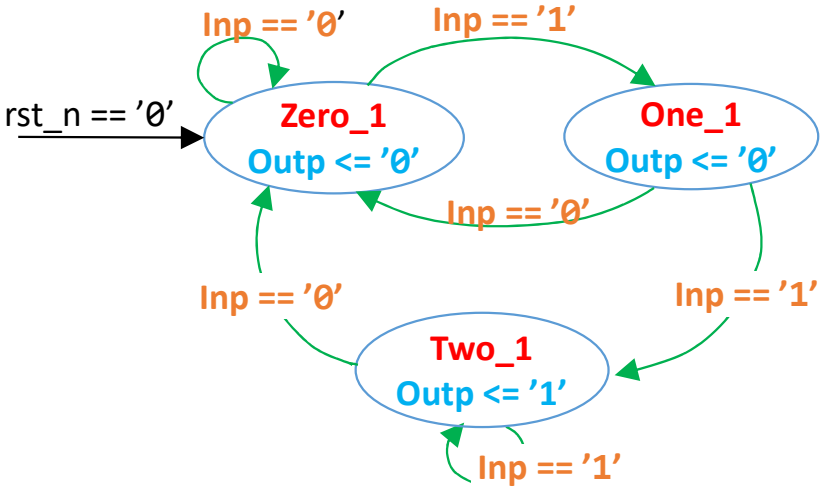


© Ahmed Hemani, KTH, Stockholm

Modelling FSMs in SystemVerilog



© Ahmed Hemani, KTH, Stockholm




```
module two_in_a_row_moore_symb(
  input  logic inp,
  input  logic clk,
  input  logic rst_n,
  output logic outp);

  // Define state type
  typedef enum logic [1:0] {
    Zero_1, One_1, Two_1
  } state_ty;

  state_ty pres_state, next_state;

  // 1. State Register Process (Sequential Logic)
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
      pres_state <= Zero_1;
    else
      pres_state <= next_state;
  end
```

```
// 2. Combinational Logic for Next-State and Output Logic
  always_comb begin
    // Default assignments (avoid latches)
    next_state = pres_state;

    case (pres_state)
      Zero_1: begin
              outp = 1'b0;
              if (!inp)
                next_state = Zero_1;
              else
                next_state = One_1;
              end

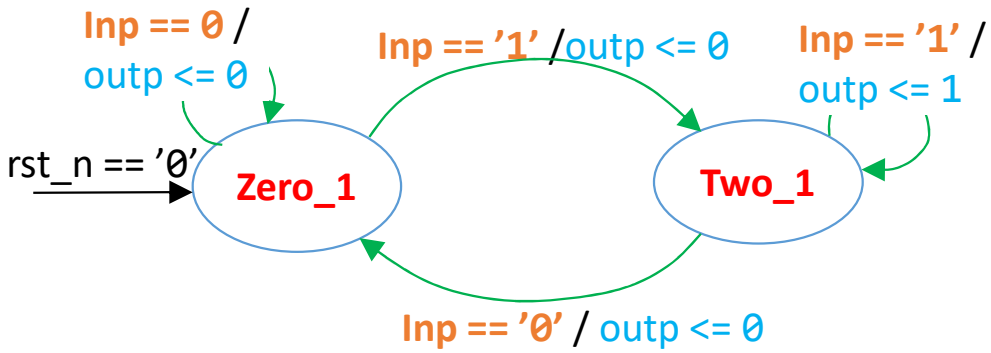
      One_1: begin
              outp = 1'b0;
              if (!inp)
                next_state = Zero_1;
              else
                next_state = Two_1;
              end

      Two_1: begin
              outp = 1'b1;
              if (!inp)
                next_state = Zero_1;
              else
                next_state = Two_1;
              end

      default: begin
              outp = 1'b0;
              next_state = Zero_1;
      end

    endcase
  end
endmodule
```

© Ahmed Hemani, KTH, Stockholm




```
module two_in_a_row_mealy_symb(input  logic inp,
                               input  logic clk,
                               input  logic rst_n,
                               output logic outp);

   // Define state type
   typedef enum  logic {Zero_1, Two_1} state_ty;

   state_ty pres_state, next_state;

   // 1. State Register Process (Sequential Logic)
   always_ff @(posedge clk or negedge rst_n) begin
      if (!rst_n)
            pres_state <= Zero_1;
      else
            pres_state <= next_state;
   end
```

```
   // 2. Combinational Logic for Next-State and Output Logic

   always_comb begin
      // Default assignments (avoid latches)
      next_state = pres_state;
      outp = 0;

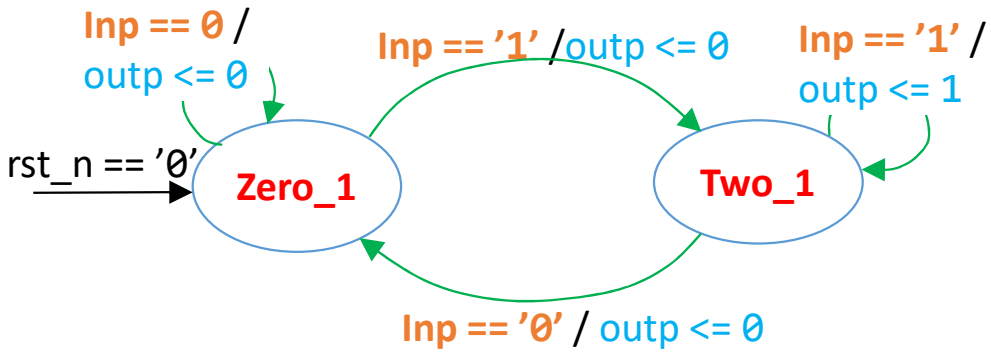
      case (pres_state)
            Zero_1: begin
               if (!inp)begin
                  next_state = Zero_1;
               end
               else begin
                  next_state = Two_1;
               end
            end

            Two_1: begin
               if (!inp) begin
                  next_state = Zero_1;
               end
               else
                 begin
                        next_state = Two_1;
                        outp = 1;
                 end
            end

            default: begin
               next_state = Zero_1;
            end

      endcase
   end
endmodule
```


© Ahmed Hemani, KTH, Stockholm




```
module two_in_a_row_mealy_reg(input  logic inp,
                              input  logic clk,
                              input  logic rst_n,
                              output logic outp);

   // Define state type
   typedef enum logic {Zero_1, Two_1} state_ty;

   state_ty pres_state, next_state;
   logic    outp_intern;


   // 1. State Register Process (Sequential Logic)
   always_ff @(posedge clk or negedge rst_n) begin
      if (!rst_n) begin
             pres_state <= Zero_1;
             outp <= 0;
      end
      else begin
             pres_state <= next_state;
             outp <= outp_intern;
      end

   end
```

```
// 2. Combinational Logic for Next-State and Output Logic

always_comb begin
   // Default assignments (avoid latches)
   next_state = pres_state;
   outp_intern = 0;

   case (pres_state)
         Zero_1: begin
            if (!inp)begin
               next_state = Zero_1;
            end
            else begin
               next_state = Two_1;
            end
         end

         Two_1: begin
            if (!inp) begin
               next_state = Zero_1;
            end
            else
              begin
                     next_state = Two_1;
                     outp_intern = 1;
              end

         end

         default: begin
            next_state = Zero_1;
         end

   endcase
end
```

© Ahmed Hemani, KTH, Stockholm

```
`timescale 1ns / 1ps

module tb_two_in_a_row;

   // Testbench signals
   logic inp;
   logic clk;
   logic rst_n;
   logic outp_moore, outp_mealy, outp_mealy_reg;

   // Instantiate the DUT (Device Under Test)
   two_in_a_row_moore_symb moore (.inp(inp),
                                  .clk(clk),
                                  .rst_n(rst_n),
                                  .outp(outp_moore));

   two_in_a_row_mealy_symb mealy (.inp(inp),
                                  .clk(clk),
                                  .rst_n(rst_n),
                                  .outp(outp_mealy));

   two_in_a_row_mealy_reg mealy_reg (.inp(inp),
                                     .clk(clk),
                                     .rst_n(rst_n),
                                     .outp(outp_mealy_reg));

   // Clock generation: 10ns clock period
   always #5 clk = ~clk;
```

```
// Test procedure
   initial begin
      // Initialize signals
      clk = 0;
      rst_n = 0;
      inp = 0;

      // Apply reset
      #10 rst_n = 1; // Release reset after 10ns

      // Stimulus
      // Apply test pattern to check FSM operation

      // Input 0 -> Output should be 0
      #10 inp = 0;

      // Input 1 -> Should go to One_1 state, output remains 0
      #10 inp = 1;

      // Input 0 agai -> Should go back to to Zero_1 state, output remains 0
      #10 inp = 0;

      // Input 1 -> Should go to one_1, output should become 0
      #10 inp = 1;

      // Input 1 -> Should go to two_1 state, output becomes 1
      #10 inp = 1;

      // Input 1 again -> Should remain in  t Two_1 state, output remains 1
      #10 inp = 1;

      // Input 0  -> Should return to Zero_1. Output becomes 0
      #10 inp = 0;

      #50 $stop;
   end
endmodule
```

© Ahmed Hemani, KTH, Stockholm